

1 问题描述

- 1.编写程序，实现一个实时时钟;
- 2.从默认时间点开始计时;
- 3.必须使用中断响应程序来处理;
- 4.计时结果(年、月、日、时、分、秒)存放在RAM的数组中;
- 5.为了调试方便，可以缩短计时时间，实现短时间内计时数天、数月、数年，要在实验报告中说明当前的计时比例;
- 6.实验报告中要附上调试结果截图

2 代码实现

```
#include "reg52.h"
//定义年月日时分秒以及每次触发中断所花的时间sec_50ms
unsigned int sec_50ms=0;
unsigned char sec=0;
unsigned char min=0;
unsigned char hour=0;
unsigned char day=1;
unsigned char month=1;
unsigned int year=2020;
unsigned char months[]={31,28,31,30,31,30,31,31,30,31,30,31};
//存放月份的天数的数组，一一对应
void timeInit(void) { //初始化定时器
    TMOD&=0xF0;
    TMOD |=0x01;
    TH0=0x3C;
    TL0=0xB0;
    ET0=1;
    EA=1;
    TR0=1;
}
void calendar(void) interrupt 1
{
    TH0=0x3C;
    TL0=0xB0;
    sec_50ms++;
    if(sec_50ms==1200){
        sec++;
        sec_50ms=0;
    }
    if(sec==60){ //秒进位分
        min++;
        sec=0;
    }
    if(min==60){ //分进位小时
        hour++;
        min=0;
    }
    if(hour==24){ //小时进位天
        day++;
        hour=0;
    }
}
```

```

        if(month!=2){
            if (day==months[month-1]+1){//如果不是二月份，直接动用月份数组里的天数
                month++;
                day=1;
            }
        }
        if(month==2){//如果是二月份，判断闰年
            if(year%4==0){
                if (day==30){
                    month++;
                    day=1;
                }
            }
            else{
                if (day==29){
                    month++;
                    day=1;
                }
            }
        }
        if(month==13){
            year++;
            month=1;
        }
        *((unsigned int *)idata *0x30)=year;
        *((unsigned char *)idata *0x32)=month;
        *((unsigned char *)idata *0x33)=day;
        *((unsigned char *)idata *0x34)=hour;
        *((unsigned char *)idata *0x35)=min;
        *((unsigned char *)idata *0x36)=sec;
        //存放到RAM中，其中只有year占双字节，月日時分秒一个字节即可满足。
    }

    void main() {
        timeInit();
        while(1) {
        }
    }
}

```

定义每隔50ms触发一次中断，由此计算TH0与TL0：

$$65536 - 50000 = 15536$$

$$15536 = 0x3CB0$$

TH0=0x3C, TL0=0xB0

用sec_50ms来计数，理论上每计20个，现实时间恰好为1s，但是在keil实际调试中发现时间有偏差，keil调试时的时间会加快，因此调整进位计数条件为1200，通过计时一分钟发现调试结果大致与现实时间相符。

默认起始时间为2020年1月1日00时00分00秒，sec_50ms起始值为0。

秒向分，分向小时的进位计数条件为60。小时向天进位计数条件为24。而对于天向月份与月份向年的进位条件有变化，因为其最小值不能为0，例如：对1月份31天，应该是day==32时令月份+1并且令day=1，而不是day==31时月份+1而day=0

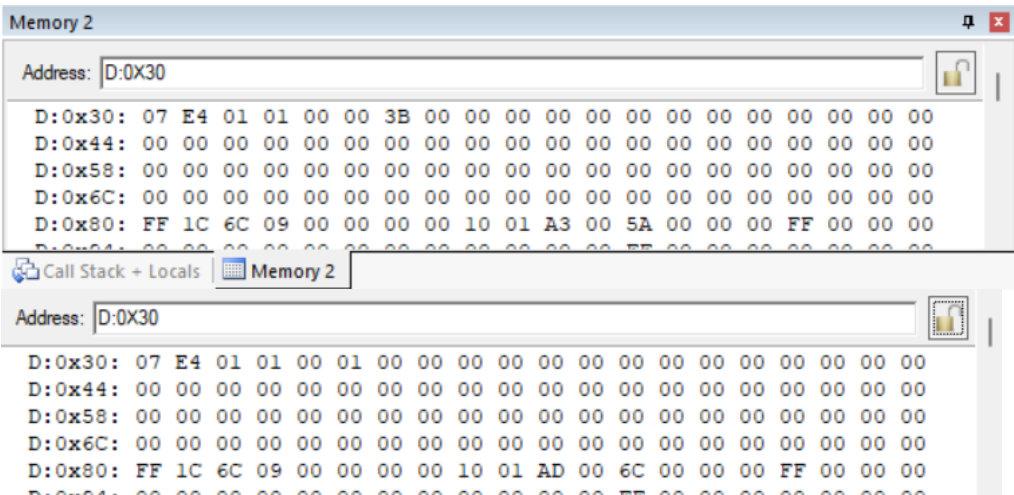
月份的进位定义了一个存储每个月天数的数组，当月份不等于2时，直接调用数组对于的天数即可，如果月份等于2，先判断是否为闰年，在选择是28还是29即可。

year是unsigned int 类型，双字节存放在RAM0x30处；sec_50ms也为unsigned int，双字节因为其值至少要1200。其余均unsigned char，单字节已经足够，按顺序从0x32开始依次存储。

3 调试结果

3.1 秒向分进位

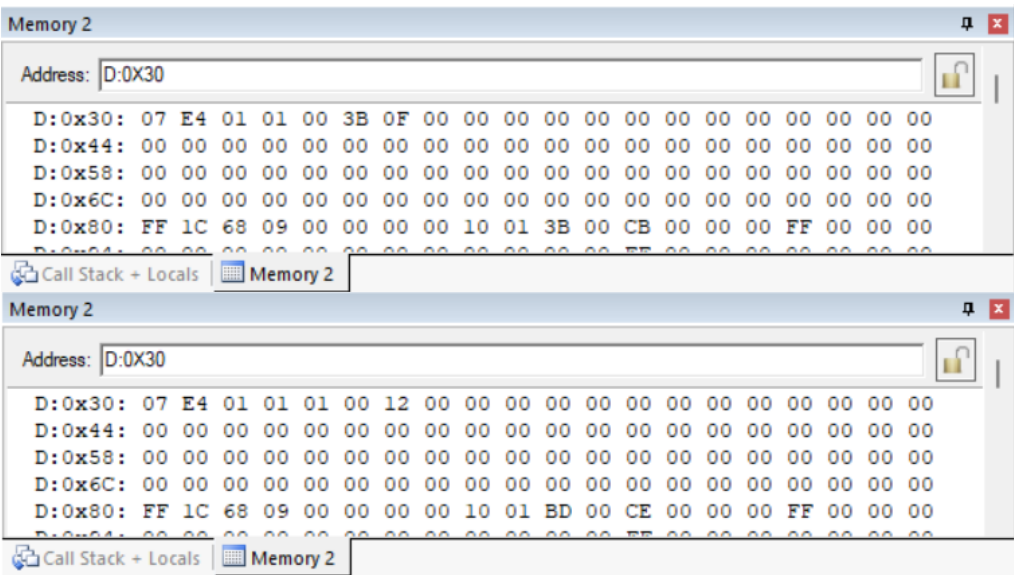
TH0=0x3C,TL0=0xB0对应中断时间为50ms，令sec_50ms计数到1200再进位，虽然理论上不等于1s，但是在keil调试时大致等于现实的1s。



秒在0x36位置，分在0x35位置。前者分为00H，秒为3BH=59；下一秒分为01H=1，秒为00H。

3.2 分向小时进位

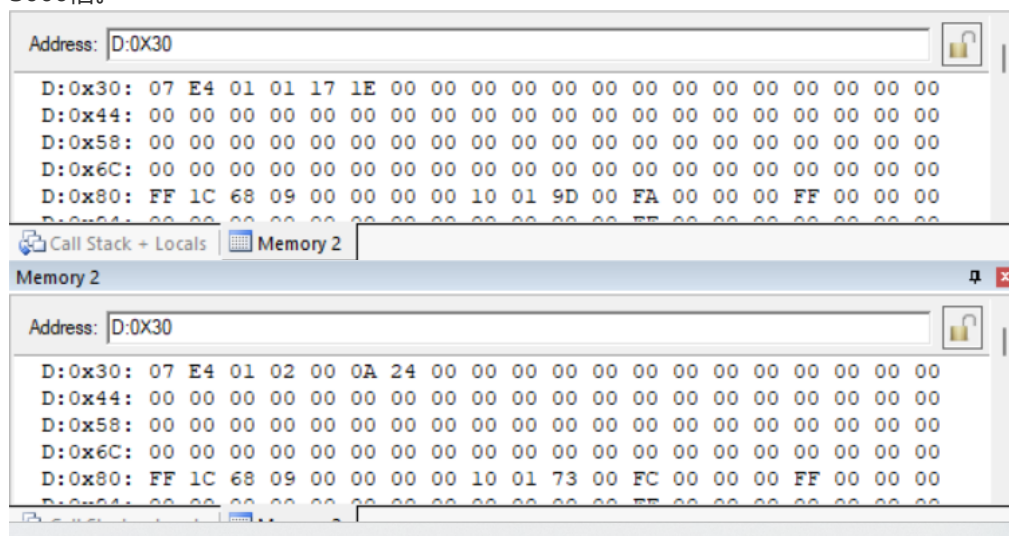
TH0=0x3C,TL0=0xB0对应中断时间为50ms，令sec_50ms计数到20再进位，与现实时间相比，增快了60倍。



分在0x35位置，时在0x34位置。前者时为00H，分为3BH=59；下一分钟时为01H=1，分为00H。

3.3 小时向天进位

TH0=0xF8, TL0=0x2C, 对应中断时间为1ms, sec_50ms计数到20再进位, 与现实时间相比, 增快了3000倍。



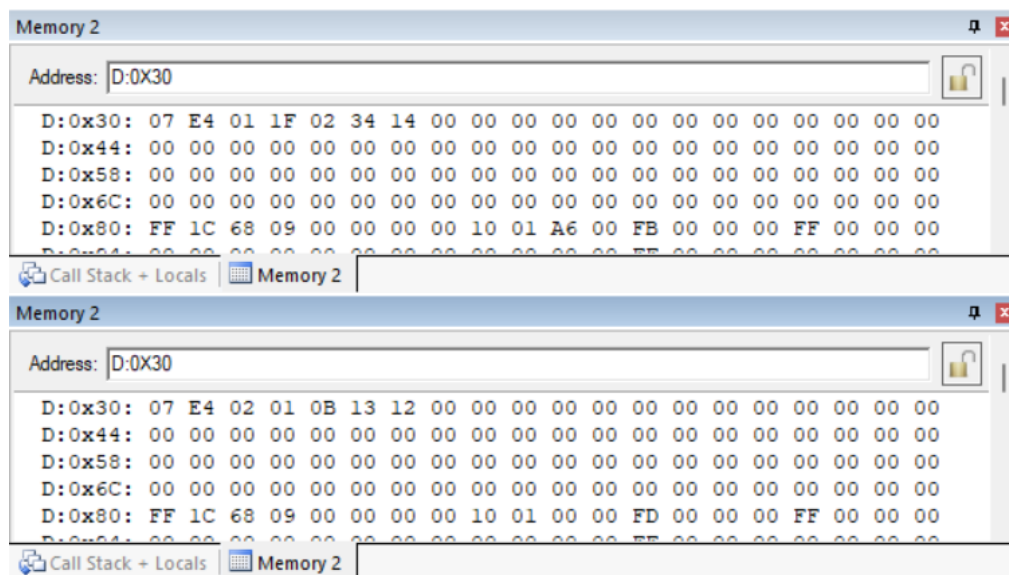
时在0x34位置，天在0x33位置。前者时为17H=23，天为01H=1；下一小时时为00H，天为02H=2。

3.4 天向月进位

3.4.1 大月

1, 3, 5, 7, 8, 10, 12月有31天, 此处选1月。

TH0=0xF8, TL0=0x2C, 对应中断时间为1ms, sec_50ms计数到2再进位, 与现实时间相比, 增快了30000倍。



天在0x33位置，月在0x32位置。前者月为01H=1，天为1FH=31；下一天月为02H=2，天为01H=1。

3.4.2 小月

4, 6, 9, 11月有30天, 此处选4月, 把默认时间改为4月。

TH0=0xF8, TL0=0x2C, 对应中断时间为1ms, sec_50ms计数到2再进位, 与现实时间相比, 增快了30000倍。


```

unsigned char month=1;
unsigned int year=2020;
unsigned char months[]={31,28,31,30,31,30,31,31,30,31,30,31};
unsigned char time[8]={0,0,0,0,0,0,0,0}; //存放显示的八位时间的编码，每两位是一组，分别对
应日，时，分，秒
unsigned char code smgduan[17]={0x3f,0x06,0x5b,0x4f,0x66,0x6d,0x7d,0x07,
                                0x7f,0x6f,0x77,0x7c,0x39,0x5e,0x79,0x71}; //显示0~F编码的数组
void timeInit(void) { //定时器初始化
    TMOD&=0xF0;
    TMOD |=0x01;
    TH0=0x3C;
    TL0=0xB0;
    ET0=1;
    EA=1;
    TR0=1;
}
void calendar(void) interrupt 1 //万年历
{
    TH0=0x3C;
    TL0=0xB0;
    sec_50ms++;
    if(sec_50ms==20){
        sec++;
        sec_50ms=0;
    }
    if(sec==60){
        min++;
        sec=0;
    }
    if(min==60){
        hour++;
        min=0;
    }
    if(hour==24){
        day++;
        hour=0;
    }
    if(month!=2){
        if (day==months[month-1]+1){
            month++;
            day=1;
        }
    }
    if(month==2){
        if(year%4==0){
            if (day==30){
                month++;
                day=1;
            }
        }
        else{
            if (day==29){
                month++;
                day=1;
            }
        }
    }
}

```

```

    }
    if(month==13){
        year++;
        month=1;
    }
    //把对应的数先转化为数码管编码在存入数组，直接可以调用输出
    time[0]=smgduan[day/10];
    time[1]=smgduan[day%10];
    time[2]=smgduan[hour/10];
    time[3]=smgduan[hour%10];
    time[4]=smgduan[min/10];
    time[5]=smgduan[min%10];
    time[6]=smgduan[sec/10];
    time[7]=smgduan[sec%10];
    *((unsigned int *)idata *0x30)=year;
    *((unsigned char *)idata *0x32)=month;
    *((unsigned char *)idata *0x33)=day;
    *((unsigned char *)idata *0x34)=hour;
    *((unsigned char *)idata *0x35)=min;
    *((unsigned char *)idata *0x36)=sec;

}

sbit LSA=P2^2;
sbit LSB=P2^3;
sbit LSC=P2^4;
void delay(unsigned int i)
{
    while(i--);
}
void DigDisplay()
{
    unsigned char i;
    for(i=0;i<8;i++)
    {
        switch(i)//确定显示的位置
        {
            case(0):
                LSA=0;LSB=0;LSC=0; break;
            case(1):
                LSA=1;LSB=0;LSC=0; break;
            case(2):
                LSA=0;LSB=1;LSC=0; break;
            case(3):
                LSA=1;LSB=1;LSC=0; break;
            case(4):
                LSA=0;LSB=0;LSC=1; break;
            case(5):
                LSA=1;LSB=0;LSC=1; break;
            case(6):
                LSA=0;LSB=1;LSC=1; break;
            case(7):
                LSA=1;LSB=1;LSC=1; break;
        }
        P0=time[i];
        delay(100); //延迟扫描时间
        P0=0x00;
    }
}

```

```

    }
}
void main()
{
    timeInit();
    while(1)
    {
        DigDisplay();
    }
}

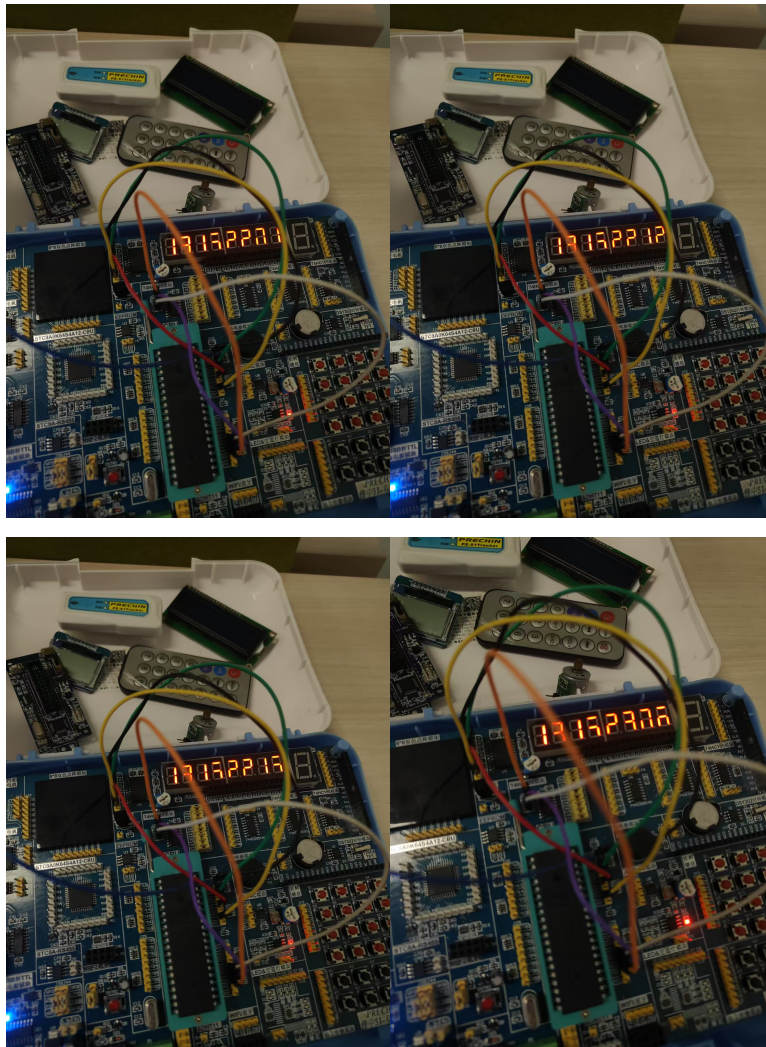
```

这时sec_50ms在计数到20时进位，此时烧录到开发板上运行时与现实时间一致。

对于日时分秒存入数组，因为都是最多两位数，那么其第一位可以用day/10表示，第二位可以用day%10表示，以此类推。又因为存放各种数字晶体管编码的数组smgduan其下标与编码代表的数字是一一对应的，因此可以直接用time[0]=smgduan[day/10]; time[1]=smgduan[day%10]; 来将处理好的编码存入time数组，直接输出time数组即可。

默认值设置为13日15时21分00秒。

效果如下：



还是可以大致区分出数字的，可惜缺杜邦线，不能完全显示。